Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

# Raport
## la
## LucrareadelaboratorNr. 4

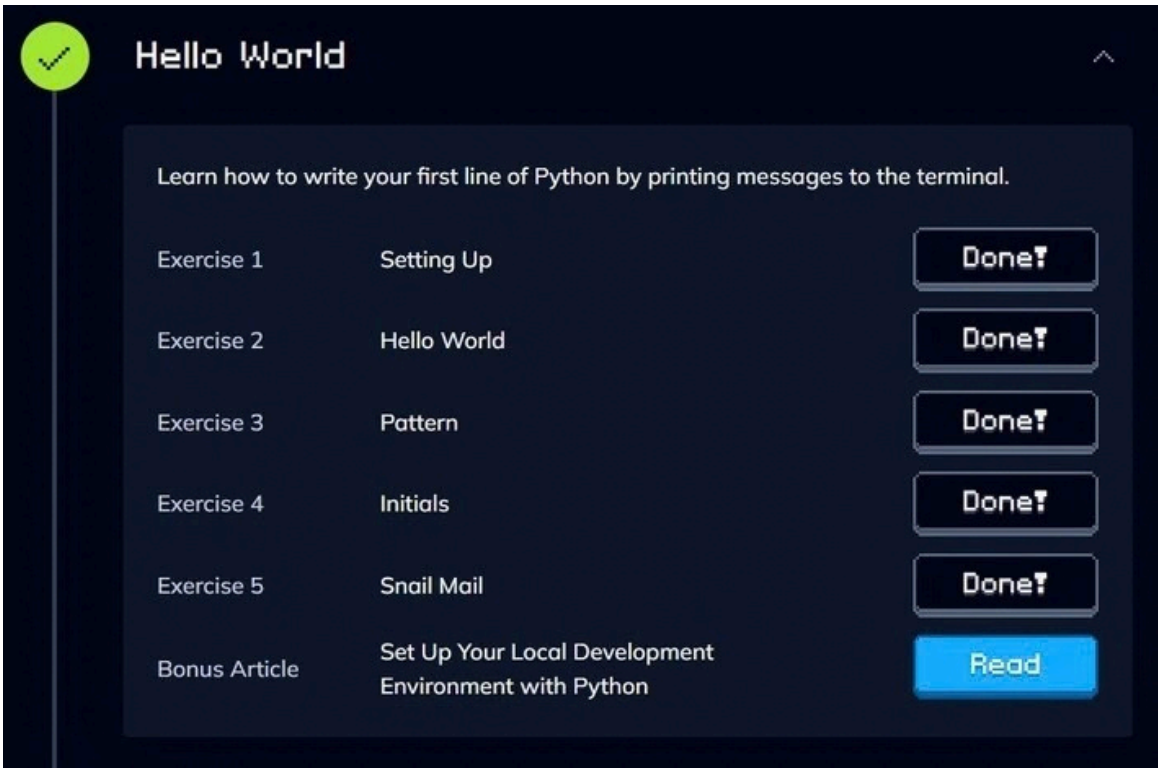
Disciplina: Tehnici de programare

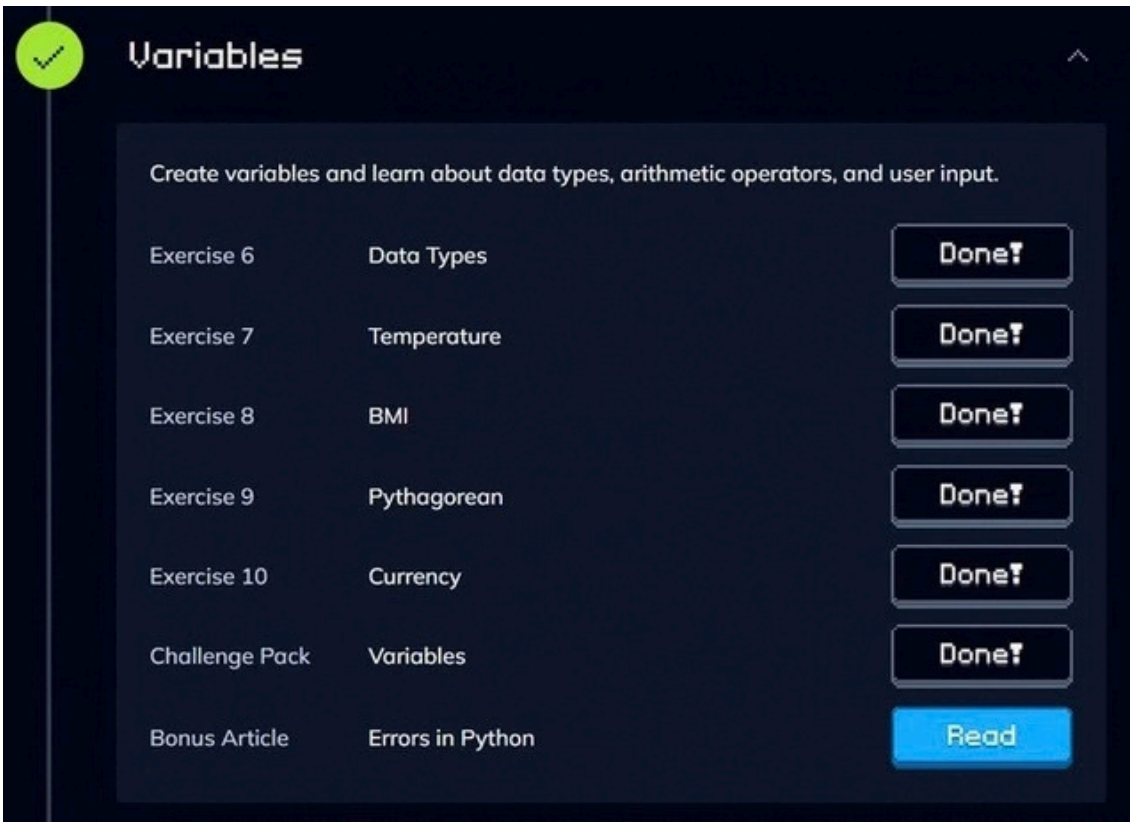
## Tema: “Python Fundamentals”

A efectuat: st. gr.SI - 263 Anastasiu Michael
A verificat: asistent universitar - Retman Patricia

# SARCINA LUCRARII

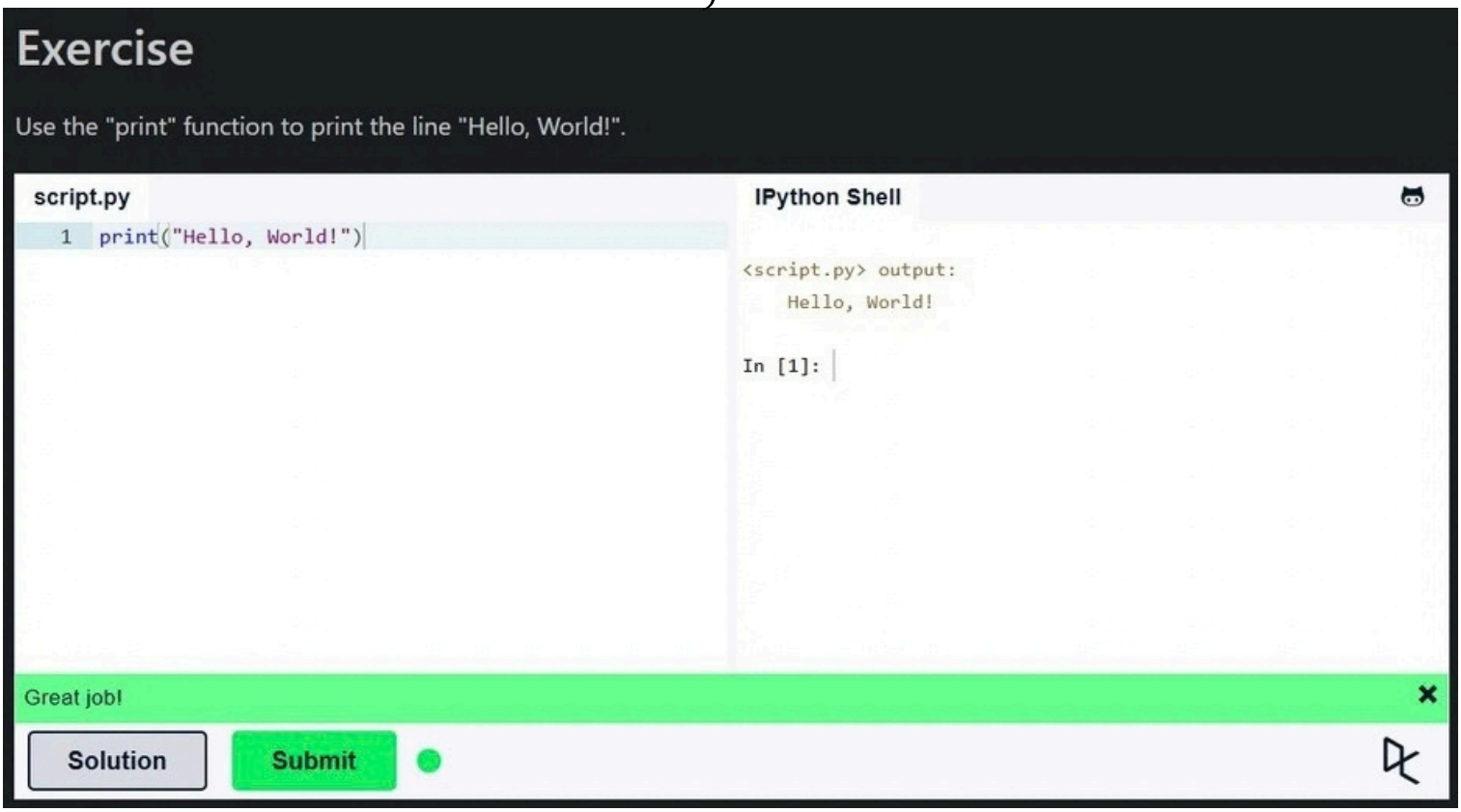
## 1 . Completarea cursului de pe platforma CODEDEX.



# 2 . Completarea cursului de pe platforma learnpython.org

- **Learn the Basics**
  - Hello, world!



- Variables and Types

## Exercise

The target of this exercise is to create a string, an integer, and a floating point number. The string should be named `mystring` and should contain the word "hello". The floating point number should be named `myfloat` and should contain the number 10.0, and the integer should be named `myint` and should contain the number 20.

**script.py**

```
1  # change this code
2  mystring = "hello"
3  myfloat = 10.0
4  myint = 20
5
6  # testing code
7  if mystring == "hello":
8      print("String: %s" % mystring)
9  if isinstance(myfloat, float) and myfloat == 10.0:
10     print("Float: %f" % myfloat)
11 if isinstance(myint, int) and myint == 20:
12     print("Integer: %d" % myint)
```

**IPython Shell**

```
<script.py> output:
    String: hello
    Float: 10.000000
    Integer: 20

In [1]:
```

Great job!

Solution | Submit

- Lists

## Exercise

In this exercise, you will need to add numbers and strings to the correct lists using the "append" list method. You must add the numbers 1,2, and 3 to the "numbers" list, and the words 'hello' and 'world' to the strings variable.

You will also have to fill in the variable second_name with the second name in the names list, using the brackets operator `[]`. Note that the index is zero-based, so if you want to access the second item in the list, its index will be 1.

**script.py**

```
1  numbers = [1,2,3]
2  strings = ['hello','world']
3  names = ["John", "Eric", "Jessica"]
4
5  # write your code here
6  second_name = names[1]
7
8
9  # this code should write out the filled arrays and
   the second name in the names list (Eric).
10 print(numbers)
11 print(strings)
12 print("The second name on the names list is %s" %
   second_name)
```

**IPython Shell**

```
<script.py> output:
    [1, 2, 3]
    ['hello', 'world']
    The second name on the names list is Eric

In [1]:
```

Great Job!

Solution | Submit

- Basic Operators

## Exercise

The target of this exercise is to create two lists called `x_list` and `y_list`, which contain 10 instances of the variables `x` and `y`, respectively. You are also required to create a list called `big_list`, which contains the variables `x` and `y`, 10 times each, by concatenating the two lists you have created.

**script.py**

```
1  x = object()
2  y = object()
3
4  # TODO: change this code
5  x_list = [x] * 10
6  y_list = [y] * 10
7  big_list = x_list + y_list
8
9  print("x_list contains %d objects" % len(x_list))
10 print("y_list contains %d objects" % len(y_list))
11 print("big_list contains %d objects" % len
   (big_list))
12
13 # testing code
14 if x_list.count(x) == 10 and y_list.count(y) ==
   10:
```

**IPython Shell**

```
    y_list contains 10 objects
    big_list contains 1 objects
    Almost there...

<script.py> output:
    x_list contains 10 objects
    y_list contains 10 objects
    big_list contains 20 objects
    Almost there...
    Great!

<script.py> output:
    x_list contains 10 objects
    y_list contains 10 objects
```

Good work!

Solution | Submit

- String Formatting

## Exercise

You will need to write a format string which prints out the data using the following syntax: `Hello John Doe. Your current balance is $53.44.`

**script.py**

```
1  data = ("John", "Doe", 53.44)
2  format_string = "Hello, %s %s. Your current balance
   is $%s."
3
4  print(format_string % data)
```

**IPython Shell**

```
<script.py> output:
    Hello, John Doe. Your current balance is $53.440000

<script.py> output:
    Hello, John Doe. Your current balance is $53.44.

In [1]:
```

Solution | Submit

- Basic String Opertaions

## Exercise

Try to fix the code to print out the correct information by changing the string.

**script.py**

```
1  s = "Strings are awesome!"
2  # Length should be 20
3  print("Length of s = %d" % len(s))
4
5  # First occurrence of "a" should be at index 8
6  print("The first occurrence of the letter a = %d"
   % s.index("a"))
7
8  # Number of a's should be 2
9  print("a occurs %d times" % s.count("a"))
10
11 # Slicing the string into bits
12 print("The first five characters are '%s'" % s[
   :5]) # Start to 5
13 print("The next five characters are '%s'" % s[5
   :10]) # 5 to 10
14 print("The thirteenth character is '%s'" % s[12])
   # Just number 12
```

**IPython Shell**

```
Length of s = 20
The first occurrence of the letter a = 8
a occurs 2 times
The first five characters are 'Strin'
The next five characters are 'gs ar'
The thirteenth character is 'a'
The characters with odd index are 'tig r wsm!'
The last five characters are 'some!'
String in uppercase: STRINGS ARE AWESOME!
String in lowercase: strings are awesome!
String starts with 'Str'. Good!
String ends with 'ome!'. Good!
Split the words of the string: ['Strings', 'are',
    'awesome!']

In [1]:
```

Solution | Submit

- Conditions

## Exercise

Change the variables in the first section, so that each if statement resolves as True.

**script.py**

```
1  # change this code
2  number = 20
3  second_number = 0
4  first_array = [1,2,3]
5  second_array = [1,2]
6
7  if number > 15:
8      print("1")
9
10 if first_array:
11     print("2")
12
13 if len(second_array) == 2:
14     print("3")
15
16 if len(first_array) + len(second_array) == 5:
```

**IPython Shell**

```
<script.py> output:
    1
    2
    3
    4
    5
    6

In [1]:
```

Great Work!

Solution | Submit

- Loops

## Exercise

Loop through and print out all even numbers from the numbers list in the same order they are received. Don't print any numbers that come after 237 in the sequence.

script.py

```
219, 390, 984, 592, 236, 105, 942, 941,
    386, 462, 47, 418, 907, 344, 236, 375, 823,
566, 597, 978, 328, 615, 953, 345,
    399, 162, 758, 219, 918, 237, 412, 566, 826,
248, 866, 950, 626, 949, 687, 217,
    815, 67, 104, 58, 512, 24, 892, 894, 767, 553
, 81, 379, 843, 831, 445, 742, 717,
    958, 609, 842, 451, 688, 753, 854, 685, 93,
857, 440, 380, 126, 721, 328, 753, 470,
    743, 527
]

# your code goes here
for number in numbers:
    if (number == 237):
        break
```

IPython Shell

```
592
236
942
386
462
418
344
236
566
978
328
162
758
918
```

Great work!

Solution | Submit

- Functions

## Exercise

In this exercise you'll use an existing function, and while adding your own to create a fully functional program.

1. Add a function named `list_benefits()` that returns the following list of strings: "More organized code", "More readable code", "Easier code reuse", "Allowing programmers to share and connect code together"
2. Add a function named `build_sentence(info)` which receives a single argument containing a string and returns a sentence starting with the given string and ending with the string " is a benefit of functions!"
3. Run and see all the functions work together!

script.py

```
# Modify this function to return a list of strings
as defined above
def list_benefits():
    return ["More organized code", "More readable
code", "Easier code reuse", "Allowing programmers
to share and connect code together"]

# Modify this function to concatenate to each
benefit - " is a benefit of functions!"
def build_sentence(benefit):
    return benefit + " is a benefit of functions!"

def name_the_benefits_of_functions():
    list_of_benefits = list_benefits()
    for benefit in list_of_benefits:
        print(build_sentence(benefit))
```

IPython Shell

```
<script.py> output:
    More organized code is a benefit of functions!
    More readable code is a benefit of functions!
    Easier code reuse is a benefit of functions!
    Allowing programmers to share and connect code together
        is a benefit of functions!

<script.py> output:
    More organized code is a benefit of functions!
    More readable code is a benefit of functions!
    Easier code reuse is a benefit of functions!
    Allowing programmers to share and connect code together
        is a benefit of functions!
```

Nice work!

Solution | Submit

- Classes and Objects

## Exercise

We have a class defined for vehicles. Create two new vehicles called car1 and car2. Set car1 to be a red convertible worth $60,000.00 with a name of Fer, and car2 to be a blue van named Jump worth $10,000.00.

**script.py**

```
8          desc_str = "%s is a %s %s worth $%.2f." %
   (self.name, self.color, self.kind, self.value)
9          return desc_str
10 # your code goes here
11 car1 = Vehicle()
12 car1.name = "Fer"
13 car1.color = "red"
14 car1.kind = "convertible"
15 car1.value = 60000
16
17 car2 = Vehicle()
18 car2.name = "Jump"
19 car2.color = "blue"
20 car2.kind = "van"
21 car2.value = 10000
22 # test code
```

**IPython Shell**

```
<script.py> output:
    Fer is a red convertible worth $60000.00.
    Jump is a blue van worth $10000.00.

In [1]:
```

Great job!

Solution | Submit

- Dictionaries

## Exercise

In this exercise, print an alphabetically sorted list of all the functions in the `re` module containing the word `find`.

**script.py**

```
1 import re
2
3 # Your code goes here
4 find_members = []
5 for member in dir(re):
6     if "find" in member:
7         find_members.append(member)
8
9 print(sorted(find_members))
```

**IPython Shell**

```
<script.py> output:
    ['findall', 'finditer']

In [1]:
```

Great work!

Solution | Submit

- Modules and Packages

## Exercise

In this exercise, print an alphabetically sorted list of all the functions in the `re` module containing the word `find`.

**script.py**

```
1  import re
2
3  # Your code goes here
4  find_members = []
5  for member in dir(re):
6      if "find" in member:
7          find_members.append(member)
8
9  print(sorted(find_members))
```

**IPython Shell**

```
<script.py> output:
    ['findall', 'finditer']

In [1]:
```

Great work!

Solution | Submit

- Input and Output

## Exercise

Complete the code below so that it reads three integers from a single line, converts each to an `int`, and prints their sum.

**script.py**

```
1  # write your code here
2  numbers = "1 2 3".split()
3  a, b, c = map(int, numbers)
4  print("sum is", a + b + c)
```

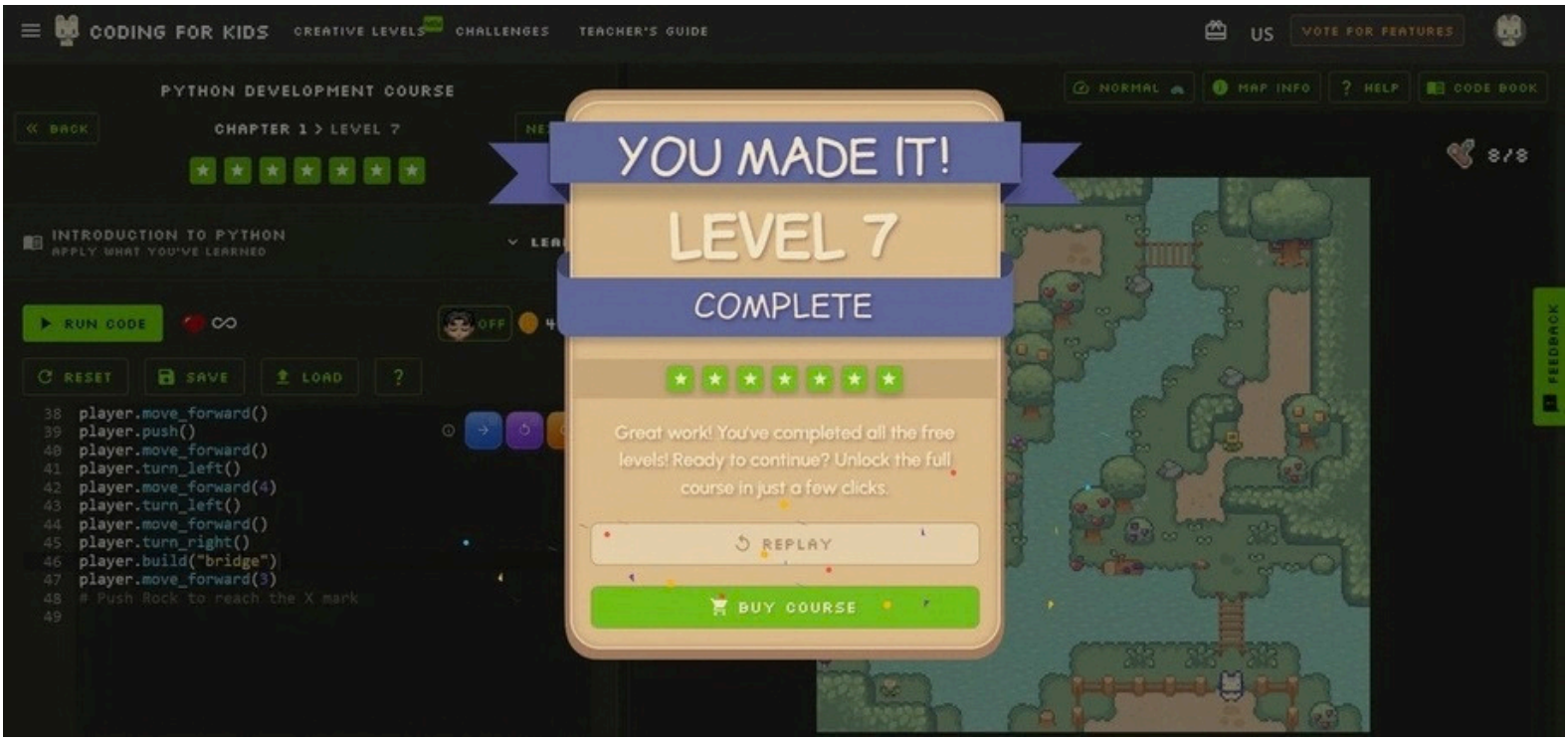
**IPython Shell**

```
DataCamp encountered the following error:
EOF when reading a line
DataCamp encountered the following error:
EOF when reading a line
DataCamp encountered the following error:
EOF when reading a line
DataCamp encountered the following error:
EOF when reading a line

In [1]:
```

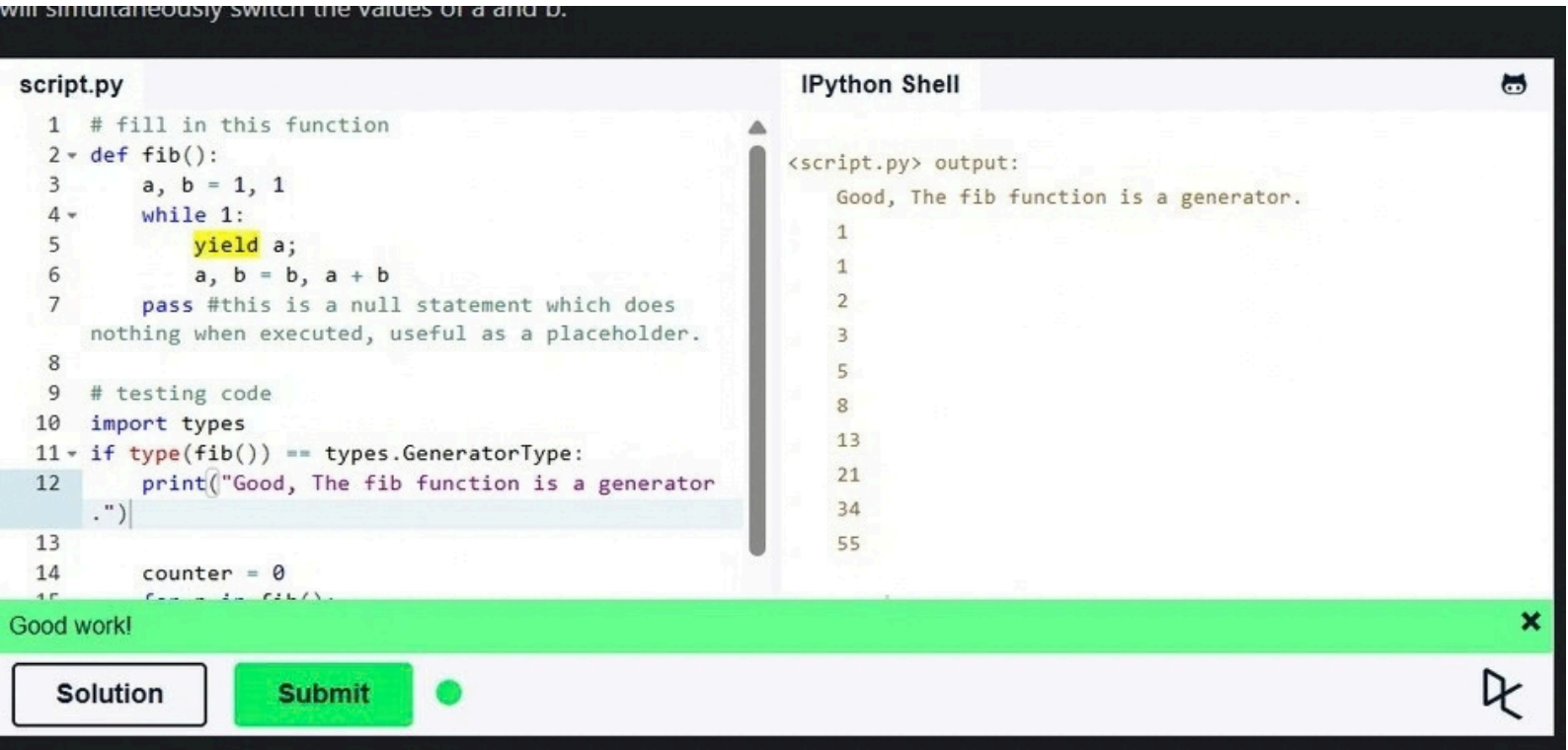
Solution | Submit | Restart Session

- **Coding for Kids**




- **Advanced Tutorials**

- Generators



- List Comprehensions

## Exercise

Using a list comprehension, create a new list called "newlist" out of the list "numbers", which contains only the positive numbers from the list, as integers.

**script.py**

```
1  numbers = [34.6, -203.4, 44.9, 68.3, -12.2, 44.6,
   12.7]
2  newlist = [int (number) for number in numbers if
   number > 0]
3  print(newlist)
```

**IPython Shell**

```
Traceback (most recent call last):
  File "script.py", line 2, in <module>
    newlist = [int (numbre) for number in numbers if number >
        0]
  File "script.py", line 2, in <listcomp>
    newlist = [int (numbre) for number in numbers if number >
        0]
NameError: name 'numbre' is not defined

<script.py> output:
    [34, 44, 68, 44, 12]

In [1]:
```

Very nice!

Solution | Submit

- Lambda functions

## Exercise

Write a program using lambda functions to check if a number in the given list is odd. Print "True" if the number is odd or "False" if not for each element.

**script.py**

```
1  l = [2,4,7,3,14,19]
2  for i in l:
3      # your code here
4      isodd = lambda number : number % 2 == 1
5      print(isodd(i))
```

**IPython Shell**

```
<script.py> output:
    False
    False
    True
    True
    False
    True

In [1]:
```

Nice work!

Solution | Submit

- Multiple Function Arguments

## Exercise

Fill in the `foo` and `bar` functions so they can receive a variable amount of arguments (3 or more) The `foo` function must return the amount of extra arguments received. The `bar` must return `True` if the argument with the keyword `magicnumber` is worth 7, and `False` otherwise.

**script.py**

```
# edit the functions prototype and implementation
def foo(a, b, c, *abc):
    return len(abc)

def bar(a, b, c, **mnum):
    return mnum["magicnumber"] == 7


# test code
if foo(1, 2, 3, 4) == 1:
    print("Good.")
if foo(1, 2, 3, 4, 5) == 2:
    print("Better.")
if bar(1, 2, 3, magicnumber=6) == False:
    print("Great.")
if bar(1, 2, 3, magicnumber=7) == True:
```

**IPython Shell**

```
<script.py> output:
    Good.
    Better.
    Great.
    Awesome!

In [1]:
```

Great work!

Solution Submit

- Regular Expressions

So, the following lines would be matched by that regex: `From: python-list@python.org To: !asp]<,. python-list@python.org`

A complete reference for the re syntax is available at the python docs.

As an example of a "proper" email-matching regex (like the one in the exercise), see this

**script.py**

```
# Exercise: make a regular expression that will match an email
import re
def test_email(your_pattern):
    pattern = re.compile(your_pattern)
    emails = ["john@example.com", "python-list@python.org", "wha.t.`1an?ug{}ly@email.com"]
    for email in emails:
        if not re.match(pattern, email):
            print("You failed to match %s" % (email))
        elif not your_pattern:
            print("Forgot to enter a pattern!")
        else:
            print("Pass")
# Your pattern here!
pattern = r"\S+@\S+"
test_email(pattern)
```

**IPython Shell**

```
<script.py> output:
    <_sre.SRE_Match object; span=(35, 39), match='[on]'>
    None
    Pass
    Pass
    Pass

In [1]:
```

Solution Submit

- Exception Handling

## Exercise

Handle all the exception! Think back to the previous lessons to return the last name of the actor.

**script.py**

```
1  # Setup
2  actor = {"name": "John Cleese", "rank": "awesome"}
3
4  # Function to modify!!!
5  def get_last_name():
6      return actor["name"].split()[1]
7
8  # Test code
9  get_last_name()
10 print("All exceptions caught! Good job!")
11 print("The actor's last name is %s" % get_last_name
   ())
```

**IPython Shell**

```
Traceback (most recent call last):
  File "script.py", line 9, in <module>
    get_last_name()
  File "script.py", line 6, in get_last_name
    return actor["last_name"].split()[1]
KeyError: 'last_name'

<script.py> output:
    All exceptions caught! Good job!
    The actor's last name is Cleese

In [1]:
```

Great work!

Solution | Submit

- Sets

In the exercise below, use the given lists to print out a set containing all the participants from event A which did not attend event B.

**script.py**

```
1  a = ["Jake", "John", "Eric"]
2  b = ["John", "Jill"]
3  A = set(a)
4  B = set(b)
5  print(A.difference(B))
```

**IPython Shell**

```
<script.py> output:
    {'Jake', 'Eric'}

In [1]:
```

Nice work!

Solution | Submit

- Serialization

The aim of this exercise is to print out the JSON string with key-value pair "Me" : 800 added to it.

script.py

```
1  import json
2
3  # fix this function, so it adds the given name
4  # and salary pair to salaries_json, and return it
5  def add_employee(salaries_json, name, salary):
6      salaries = json.loads(salaries_json)
7      # Add your code here
8      salaries[name] = salary
9      return json.dumps(salaries)
10
11 # test code
12 salaries = '{"Alfred" : 300, "Jane" : 400 }'
13 new_salaries = add_employee(salaries, "Me", 800)
14 decoded_salaries = json.loads(new_salaries)
15 print(decoded_salaries["Alfred"])
16 print(decoded_salaries["Jane"])
```

IPython Shell

```
<script.py> output:
    300
    400
    800

In [1]:
```

Great work!

Solution | Submit

- Partial functions

# Exercise

Edit the function provided by calling partial() and replacing the first three variables in func(). Then print with the new partial function using only one input variable so that the output equals 60.

script.py

```
1  #Following is the exercise, function provided:
2  from functools import partial
3  def func(u, v, w, x):
4      return u*4 + v*3 + w*2 + x
5  #Enter your code here to create and print with your
   partial function
6  tst = partial(func, 5, 6, 7)
7  print(tst(8))
```

IPython Shell

```
<script.py> output:
    60

In [1]:
```

Good job!

Solution | Submit

# CONCLUZIE

Inurma parcurgeriiresurselor de invatare **Codexdex** si **learnpython.org** am asimilat baza solida in programare, acoperind concepte fundamentale precum variabile, fluxul de control si buclele, dar si notiuni avansate precum functii lambda, generatori si gestionarea exceptiilor. Combinarea teoriei cu exercitiile interactive m-a ajutat sa inteleg mai bine limbajul Python si sa exersez practic.